

Encoding for Objects Matters

Dave Mason

Toronto Metropolitan University

©2025 Dave Mason



Toronto
Metropolitan
University

Encoding for Objects Matters - Redux

Dave Mason and Nishil Kapedia
Toronto Metropolitan University

©2026 Dave Mason



Toronto
Metropolitan
University

Efficient Encoding of Objects for Dynamic-Typing

Dave Mason and Nishil Kapedia
Toronto Metropolitan University

©2026 Dave Mason



Toronto
Metropolitan
University

- untyped
 - assembler, BCPL, B
 - only machine instruction controls interpretation of bits

- untyped
 - assembler, BCPL, B
 - only machine instruction controls interpretation of bits
- static typing
 - C, Rust, Zig, C++ Java
 - variables, functions, expressions have type
 - compile-time overload/operator polymorphism
 - allows operators like +, -, *, and others to behave differently based on the data types of their operands

- untyped
 - assembler, BCPL, B
 - only machine instruction controls interpretation of bits
- static typing
 - C, Rust, Zig, C++ Java
 - variables, functions, expressions have type
 - compile-time overload/operator polymorphism
 - allows operators like +, -, *, and others to behave differently based on the data types of their operands
- dynamic typing
 - Smalltalk, Python, JavaScript
 - values, expressions have type
 - run-time overload/operator polymorphism

Encoding Considerations for Objects

- many implementation, from byte interpreters to threaded execution to JIT'ed native code

Encoding Considerations for Objects

- many implementation, from byte interpreters to threaded execution to JIT'ed native code
- encoding choices will affect all of the implementations, to varying degrees

Encoding Considerations for Objects

- many implementation, from byte interpreters to threaded execution to JIT'ed native code
- encoding choices will affect all of the implementations, to varying degrees
- **Determining types**
 - operations must be parameterized at run time by the types of the values
 - the first step is to access the types of the values - may slow pipeline
 - types in memory can cause a pipeline stall until they can be loaded
 - may be shifting/masking operations required to access types

Encoding Considerations for Objects

- many implementation, from byte interpreters to threaded execution to JIT'ed native code
- encoding choices will affect all of the implementations, to varying degrees
- **Determining types**
 - operations must be parameterized at run time by the types of the values
 - the first step is to access the types of the values - may slow pipeline
 - types in memory can cause a pipeline stall until they can be loaded
 - may be shifting/masking operations required to access types
- **Accessing values**
 - In order to perform operations, the values must be made available to the CPU.
 - accessing values in memory is an order of magnitude slower than accessing them in registers
 - even if the values happen to be in cache they will be several times slower than in registers
 - may be shifting/masking operations required to access values

Encoding Considerations for Objects

- many implementation, from byte interpreters to threaded execution to JIT'ed native code
- encoding choices will affect all of the implementations, to varying degrees
- **Determining types**
 - operations must be parameterized at run time by the types of the values
 - the first step is to access the types of the values - may slow pipeline
 - types in memory can cause a pipeline stall until they can be loaded
 - may be shifting/masking operations required to access types
- **Accessing values**
 - In order to perform operations, the values must be made available to the CPU.
 - accessing values in memory is an order of magnitude slower than accessing them in registers
 - even if the values happen to be in cache they will be several times slower than in registers
 - may be shifting/masking operations required to access values
- **Supporting Memory Management**
 - dynamic languages invariably have automatic memory management, (reference counting or garbage collection)
 - objects need to be allocated, and removed when no longer needed
 - both actions have a significant cost, and the more objects allocated in memory, the greater the cost

- but first, a detour into floating point

- but first, a detour into floating point
- 2015 - Miranda and Béra - SPUR encoding

- but first, a detour into floating point
- 2015 - Miranda and Béra - SPUR encoding
- 2026 - Gilot, Wrigstad and Darulova - 62% of github code uses floating point

- but first, a detour into floating point
- 2015 - Miranda and Béra - SPUR encoding
- 2026 - Gilot, Wrigstad and Darulova - 62% of github code uses floating point
- 2023 - Zurstrassen, Reimann, Bosbach, Juenger and Leupers - *Evaluation of the RISC-V Floating Point Extensions*,

- but first, a detour into floating point
- 2015 - Miranda and Béra - SPUR encoding
- 2026 - Gilot, Wrigstad and Darulova - 62% of github code uses floating point
- 2023 - Zurstrassen, Reimann, Bosbach, Juenger and Leupers - *Evaluation of the RISC-V Floating Point Extensions*,
- 2025 - Melançon, Serrano and Feeley - *Float Self-Tagging* - for Scheme

- but first, a detour into floating point
- 2015 - Miranda and Béra - SPUR encoding
- 2026 - Gilot, Wrigstad and Darulova - 62% of github code uses floating point
- 2023 - Zurstrassen, Reimann, Bosbach, Juenger and Leupers - *Evaluation of the RISC-V Floating Point Extensions*,
- 2025 - Melançon, Serrano and Feeley - *Float Self-Tagging* - for Scheme
- can choose to encode some float values as immediate, rest in memory

...Float encoding...

Exponent	Range	Freqncy	spur	..NZ	fst1	fst2	fst4	zag6	NaN
000000000000	0	0.04%	✓	✗	✓	✓	✓	✓	✓
00000	2.05×10^{-289}	0.06%	✗	✗	✓	✓	✓	✓	✓
00001	3.79×10^{-270}	1.17%	✗	✗	✗	✓	✓	✓	✓
0001	1.29×10^{-231}	1.19%	✗	✗	✗	✗	✓	✓	✓
001-010	1.73×10^{-77}	0.89%	✗	✗	✗	✗	✗	✓	✓
0110	5.88×10^{-39}	0.69%	✗	✗	✗	✗	✓	✓	✓
01110	1.08×10^{-19}	1.42%	✓	✓	✗	✓	✓	✓	✓
01111-10000	3.69×10^{19}	94.51%	✓	✓	✓	✓	✓	✓	✓
10001	6.81×10^{38}	0.01%	✓	✓	✗	✓	✓	✓	✓
1001	2.32×10^{77}	0.01%	✗	✗	✗	✗	✓	✓	✓
101-110	3.11×10^{231}	0.00%	✗	✗	✗	✗	✗	✗	✓
1110	1.06×10^{270}	0.00%	✗	✗	✗	✗	✓	✓	✓
11110	1.95×10^{289}	0.00%	✗	✗	✗	✓	✓	✓	✓
11111	nan	0.04%	✗	✗	✓	✓	✓	✓	✓

Encoding	Encode	Unencodable	Decode	Coverage	Tags Used
spur	0.851ns	0.989ns	0.609ns	95.98%	4 (5,6,7 reserved)
spurNZ	0.622ns	0.619ns	0.619ns	95.94%	4 (5,6,7 reserved)
fst1	0.595ns	0.586ns	0.482ns	94.61%	4 (5,6,7 reserved)
fst2	0.593ns	0.585ns	0.497ns	97.21%	4,5 (6,7 reserved)
fst4	0.711ns	0.640ns	0.497ns	99.11%	2,3,6,7
zag6	0.489ns	0.450ns	0.488ns	100.00%	2,3,4,5,6,7
nan	0.396ns	—	0.457ns	100.00%	N/A
nun	0.404ns	—	0.405ns	100.00%	N/A

Catalog of Encodings

- all mutable object need to be in memory
- many possible ways to encode the range of immutable values that arise in normal calculations
- we discuss 5 possible encodings that encompass the range from all values being in memory, to treating as many things as immediate as possible
- we examine each from the perspective of the three considerations

- all mutable object need to be in memory
- many possible ways to encode the range of immutable values that arise in normal calculations
- we discuss 5 **17** possible encodings that encompass the range from all values being in memory, to treating as many things as immediate as possible
- we examine each from the perspective of the three considerations

Every Object in Memory

- This is the simplest model.
- Determining the value type requires a memory access, as does accessing the actual value. Interoperation with foreign functions is complicated.
- The result of every operation must be allocated to memory.
- This will create a huge amount of churn, although most of the values will be very short-lived.
- Immutable values `nil` , `true` , `false` , and ASCII Characters are allocated at fixed addresses (Special Objects array)

Every Object in Memory with SmallInteger Cache

- This is a simple optimization by statically defining a set of small integers (say -5..100).
- This means that if a result is in this pre-allocated set, then no allocation or collection is required.

- Early Lisp and Smalltalk implementers noticed that the vast majority of values that were created were for *SmallInteger* objects.
- Most such systems tag small integers with a 1 bit tag (either in the low bit leaving all other addresses natural by aligning all objects on at least a word boundary, or (less commonly) using the sign bit).
- A small drawback of this encoding is that 1 bit of precision is lost, but most such systems move automatically to big integers with unbounded precision on overflow and on modern, 64-bit systems that one bit of precision is fairly irrelevant.

- Miranda/Bera created the SPUR encoding
- cleverly extends the 1-bit tag to 3 bits and in addition to `SmallInteger` and general (memory) objects, supports encoding for Unicode characters and a subset of `Float`.
- Examining the low 3 bits of an object, a coding of 0 is a pointer to a memory object, 1 is a 61-bit `SmallInteger`, 2 is a Unicode `Character`, and 4 is a `Float`.
- Only the pointer tag is a natural value - all the others require some decoding before use.
- As with tagging reducing precision on integers, the Spur encoding limits the range of floating point values.

- NaN encoding utilizes the large number of code points in the IEEE-7544 floating point encoding that do not represent valid floating-point numbers, by using those bit patterns to represent values of other types, including pointers and *SmallInteger* .
- This encoding has been used by Spidermonkey and was the originally planned encoding for Zag Smalltalk
- This encoding supports, within 64 bits, all *Float* values naturally, as well as 51-bit *SmallInteger* values, pointers to memory objects, *nil* , booleans, symbols, characters, as well as several common *BlockClosure* values.

Tag Compact Classes

- there are a set of about 8 encodings that use high bits to encode 32 classes
- we only show a few of them here - they led to...

Tag Most Possible Values

- Zag Smalltalk now uses an encoding using both low and high tags.
- currently uses FST2, but could use FST4 or FST6
- It uses the bottom 3 bits of an object: 2 encodes 62-bit *SmallIntegers* ; 4 encodes floating point (and uses 1 as well)
- everything else uses a 5-bit high tag to represent 32 other classes
- one of those represents a pointer to a heap object
- 0 represents a native (C) pointer - including NULL (*UndefinedObject*)
- the immediate classes include booleans, symbols, Unicode *Character* .
- the immediate classes also include 13 kinds of special block closures, including some non-local returns, that are common in existing code.

Experiment

- we implemented all 17 of these encodings
- initial experiment with integer and float fibonacci
- have inlined and non-inlined versions - with *BlockClosures*
- exercises the integer and floating-point paths, and the version with no inlining exercises the *BlockClosure* creation and non-local return
- unfortunately, don't yet have results

- we implemented all 17 of these encodings
- initial experiment with integer and float fibonacci
- have inlined and non-inlined versions - with *BlockClosures*
- exercises the integer and floating-point paths, and the version with no inlining exercises the *BlockClosure* creation and non-local return
- ~~unfortunately, don't yet have results~~ **we have results**

Hypothesis

- **Every Object in Memory**

- The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers

Hypothesis

- **Every Object in Memory** ✓
 - The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers
- **Every Object in Memory with SmallInteger Cache**
 - The first 11 fibonacci numbers are in the cache we pre-allocated, and those are the most frequently generated values, so the integer versions of the benchmark should run significantly faster.

Hypothesis

- **Every Object in Memory** ✓
 - The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers
- **Every Object in Memory with SmallInteger Cache** ✓
 - The first 11 fibonacci numbers are in the cache we pre-allocated, and those are the most frequently generated values, so the integer versions of the benchmark should run significantly faster.
- **Tag** *SmallInteger*
 - Now all integers (for the test) have immediate values, so the integer tests should speed up significantly.
 - The floating-point version will have no speedup.

Hypothesis

- **Every Object in Memory** ✓
 - The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers
- **Every Object in Memory with SmallInteger Cache** ✓
 - The first 11 fibonacci numbers are in the cache we pre-allocated, and those are the most frequently generated values, so the integer versions of the benchmark should run significantly faster.
- **Tag** *SmallInteger* ✓
 - Now all integers (for the test) have immediate values, so the integer tests should speed up significantly.
 - The floating-point version will have no speedup.
- **Tag** *SmallInteger* , *Character* , *Float*
 - Because SPUR encodes immediate values for all of the generated floating-point values, the *Float* version of this should run significantly faster.
 - The integer version should experience no speedup.
 - all of the above should be noticeably slower for the unoptimized version

Hypothesis

- **Every Object in Memory** ✓
 - The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers
- **Every Object in Memory with SmallInteger Cache** ✓
 - The first 11 fibonacci numbers are in the cache we pre-allocated, and those are the most frequently generated values, so the integer versions of the benchmark should run significantly faster.
- **Tag** *SmallInteger* ✓
 - Now all integers (for the test) have immediate values, so the integer tests should speed up significantly.
 - The floating-point version will have no speedup.
- **Tag** *SmallInteger* , *Character* , *Float* ✓
 - Because SPUR encodes immediate values for all of the generated floating-point values, the *Float* version of this should run significantly faster.
 - The integer version should experience no speedup.
 - all of the above should be noticeably slower for the unoptimized version
- **NaN Encoding**
 - NaN encoding has the floating-point already in the correct format, therefore this should have the fastest floating point version
 - integer versions will be somewhat slower (more manipulation is required to retrieve the integral value)
 - Because the blocks are immediate values, the “no-inlining” version should be comparatively faster

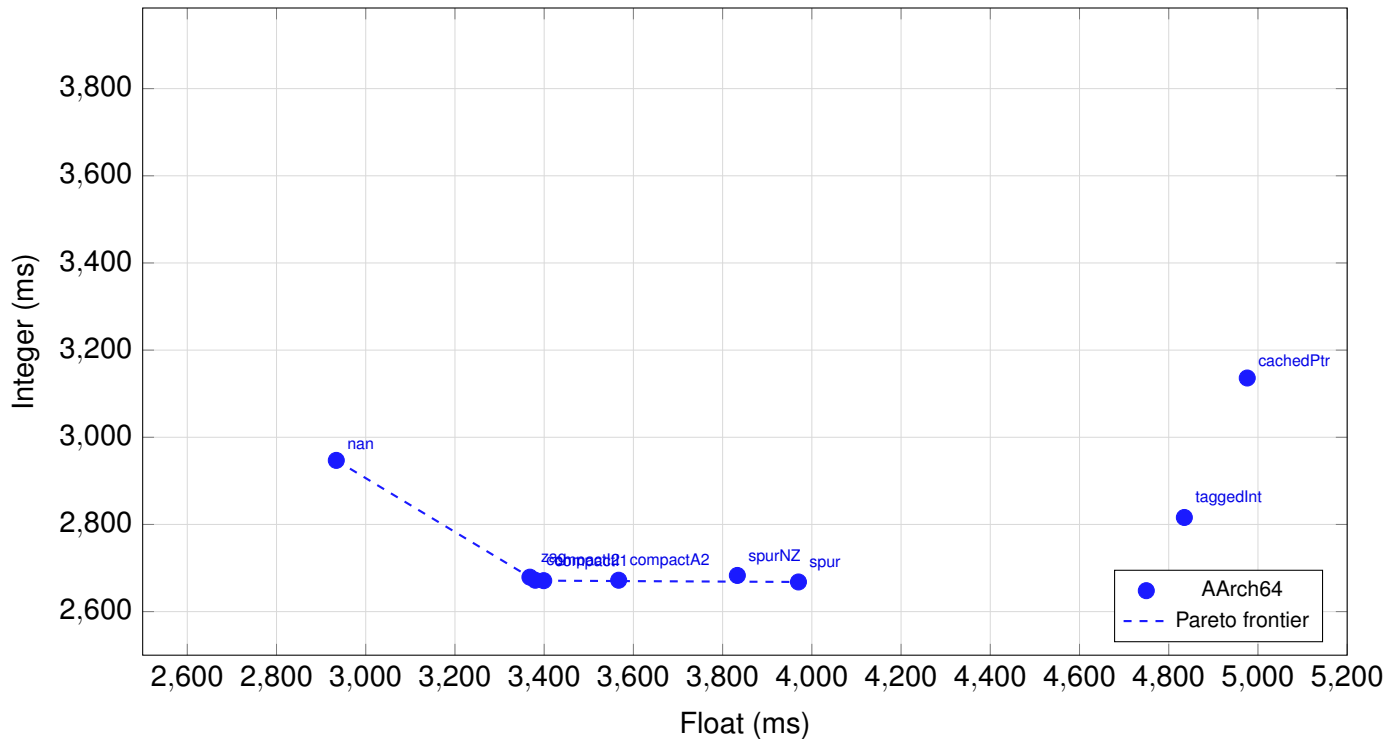
Hypothesis

- **Every Object in Memory** ✓
 - The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers
- **Every Object in Memory with SmallInteger Cache** ✓
 - The first 11 fibonacci numbers are in the cache we pre-allocated, and those are the most frequently generated values, so the integer versions of the benchmark should run significantly faster.
- **Tag *SmallInteger*** ✓
 - Now all integers (for the test) have immediate values, so the integer tests should speed up significantly.
 - The floating-point version will have no speedup.
- **Tag *SmallInteger* , *Character* , *Float*** ✓
 - Because SPUR encodes immediate values for all of the generated floating-point values, the *Float* version of this should run significantly faster.
 - The integer version should experience no speedup.
 - all of the above should be noticeably slower for the unoptimized version
- **NaN Encoding** ✗
 - NaN encoding has the floating-point already in the correct format, therefore this should have the fastest floating point version
 - integer versions will be somewhat slower (more manipulation is required to retrieve the integral value)
 - Because the blocks are immediate values, the “no-inlining” version should be comparatively faster
- **Tag *Most Possible Values***
 - Zag native format may be very slightly slower than Spur for inlined integer
 - should also be a bit faster than Spur for floating point, because the decoding is a couple of instructions faster
 - Again, the blocks are immediate values, the “no-inlining” version should much faster

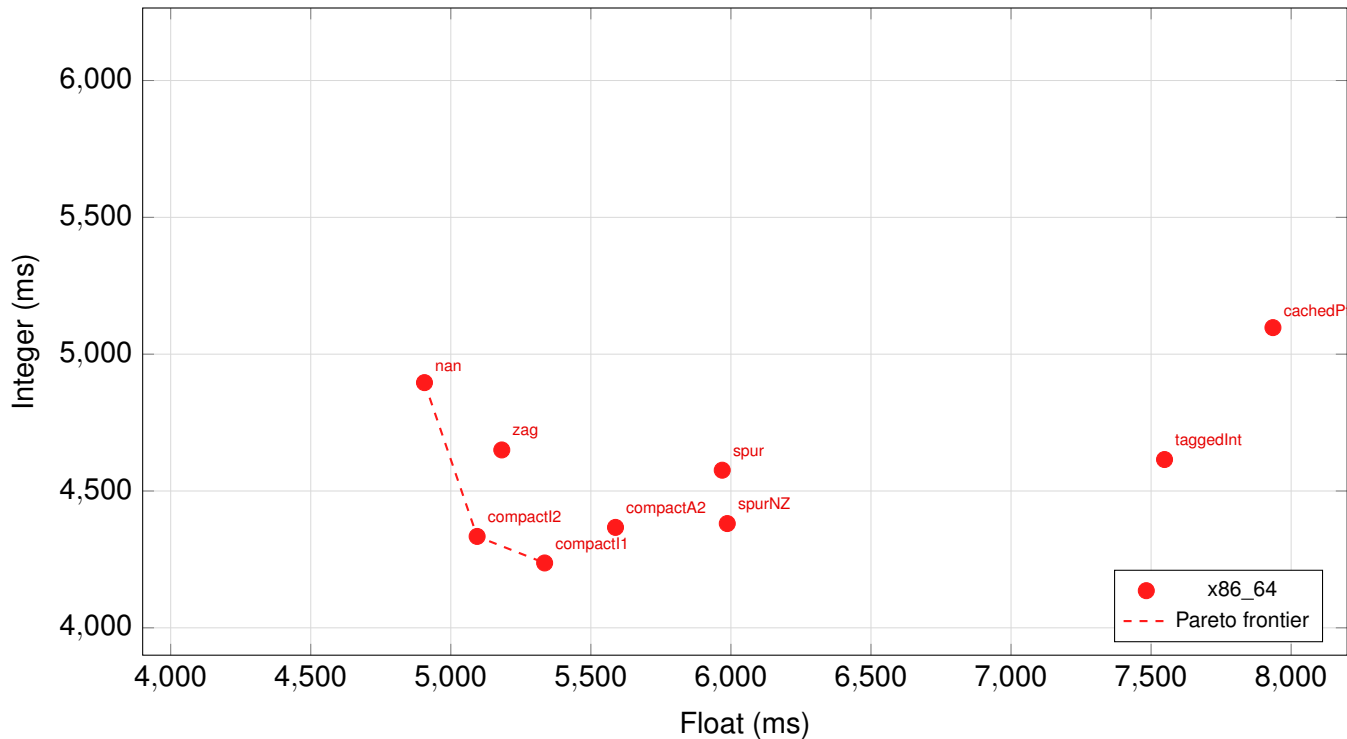
Hypothesis

- **Every Object in Memory** ✓
 - The result of every addition will allocate in the nursery, which will create a great deal of churn for the numbers
- **Every Object in Memory with SmallInteger Cache** ✓
 - The first 11 fibonacci numbers are in the cache we pre-allocated, and those are the most frequently generated values, so the integer versions of the benchmark should run significantly faster.
- **Tag *SmallInteger*** ✓
 - Now all integers (for the test) have immediate values, so the integer tests should speed up significantly.
 - The floating-point version will have no speedup.
- **Tag *SmallInteger* , *Character* , *Float*** ✓
 - Because SPUR encodes immediate values for all of the generated floating-point values, the *Float* version of this should run significantly faster.
 - The integer version should experience no speedup.
 - all of the above should be noticeably slower for the unoptimized version
- **NaN Encoding** ✗
 - NaN encoding has the floating-point already in the correct format, therefore this should have the fastest floating point version
 - integer versions will be somewhat slower (more manipulation is required to retrieve the integral value)
 - Because the blocks are immediate values, the “no-inlining” version should be comparatively faster
- **Tag *Most Possible Values*** ✓
 - Zag native format may be very slightly slower than Spur for inlined integer
 - should also be a bit faster than Spur for floating point, because the decoding is a couple of instructions faster
 - Again, the blocks are immediate values, the “no-inlining” version should much faster

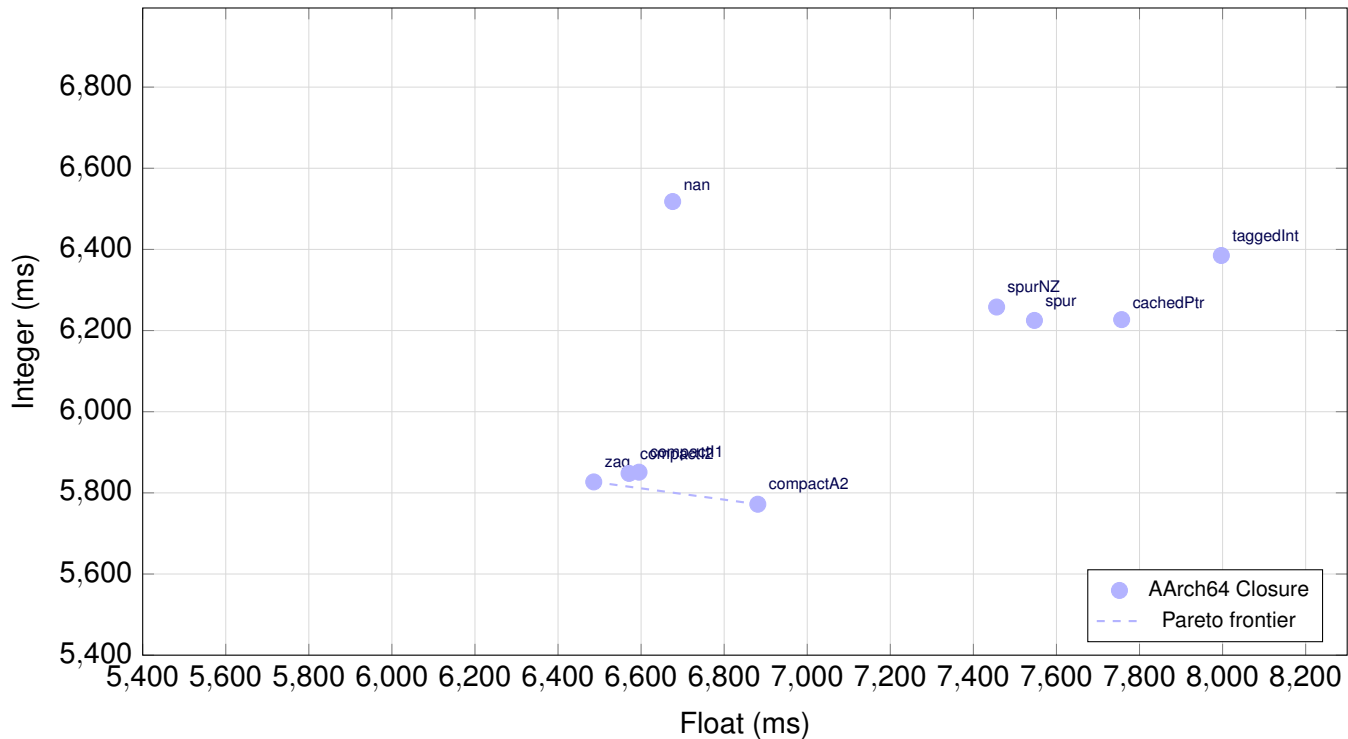
Integer vs Float - AARCH64



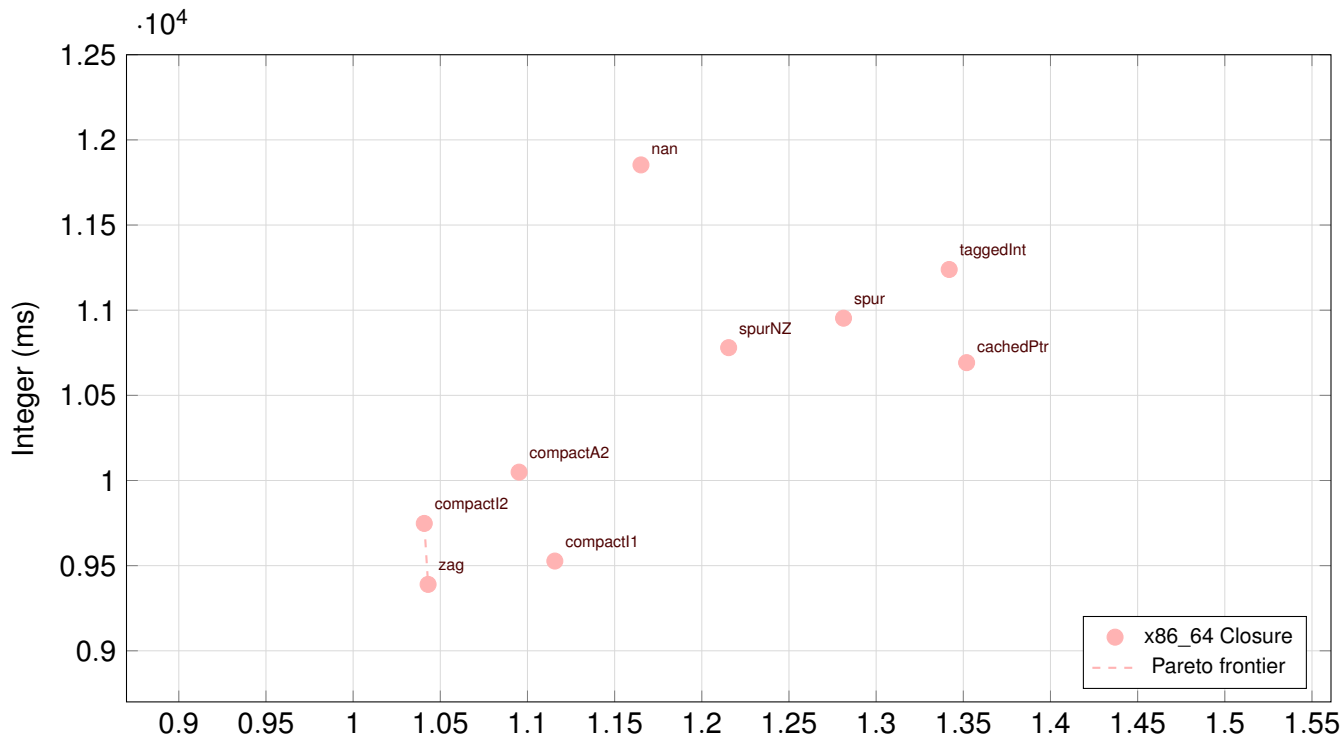
Integer vs Float - x86-64



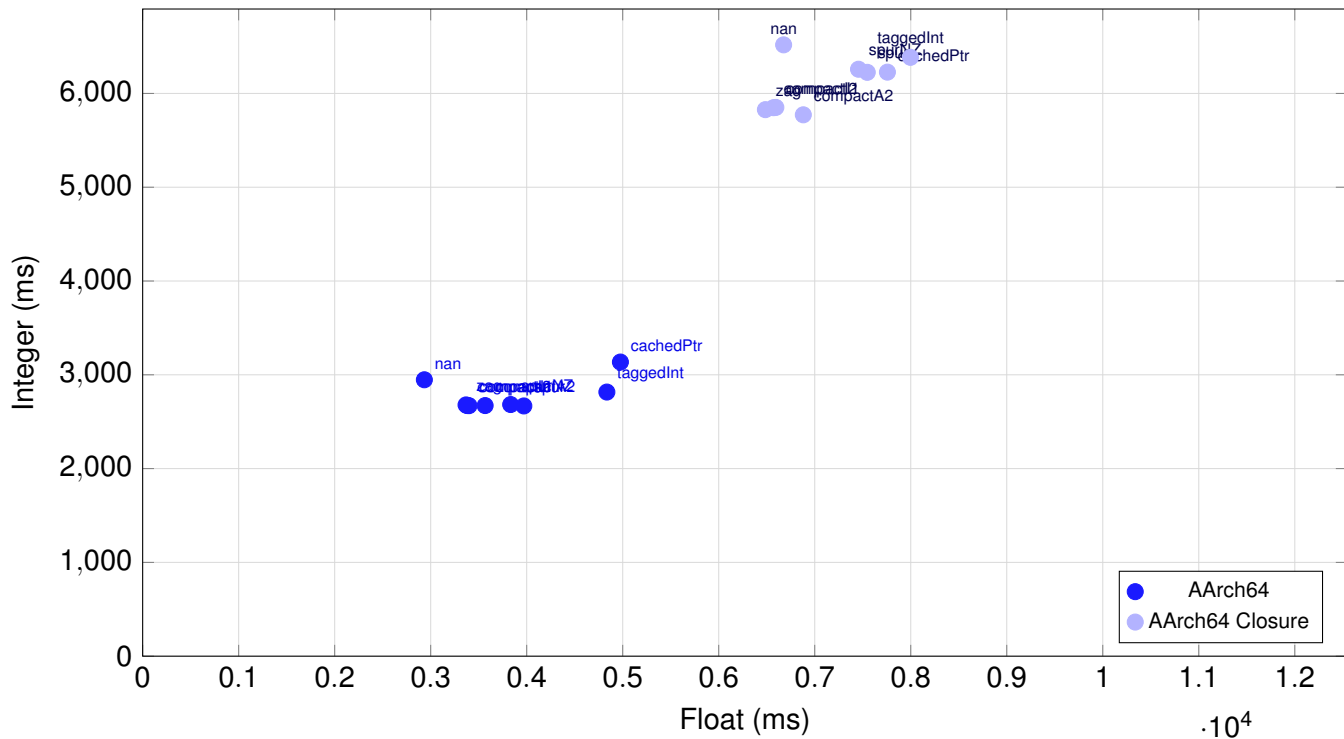
Integer vs Float - BlockClosures - AARCH64



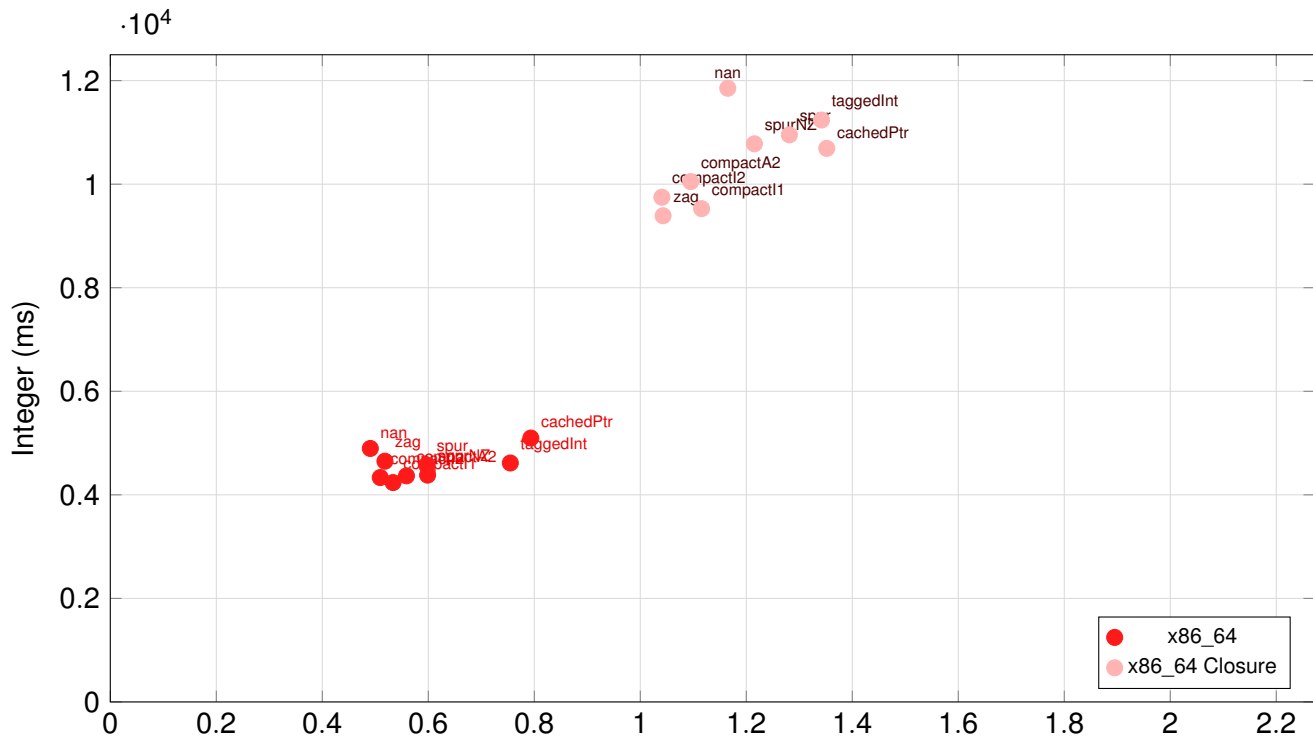
Integer vs Float - BlockClosures - X86-64



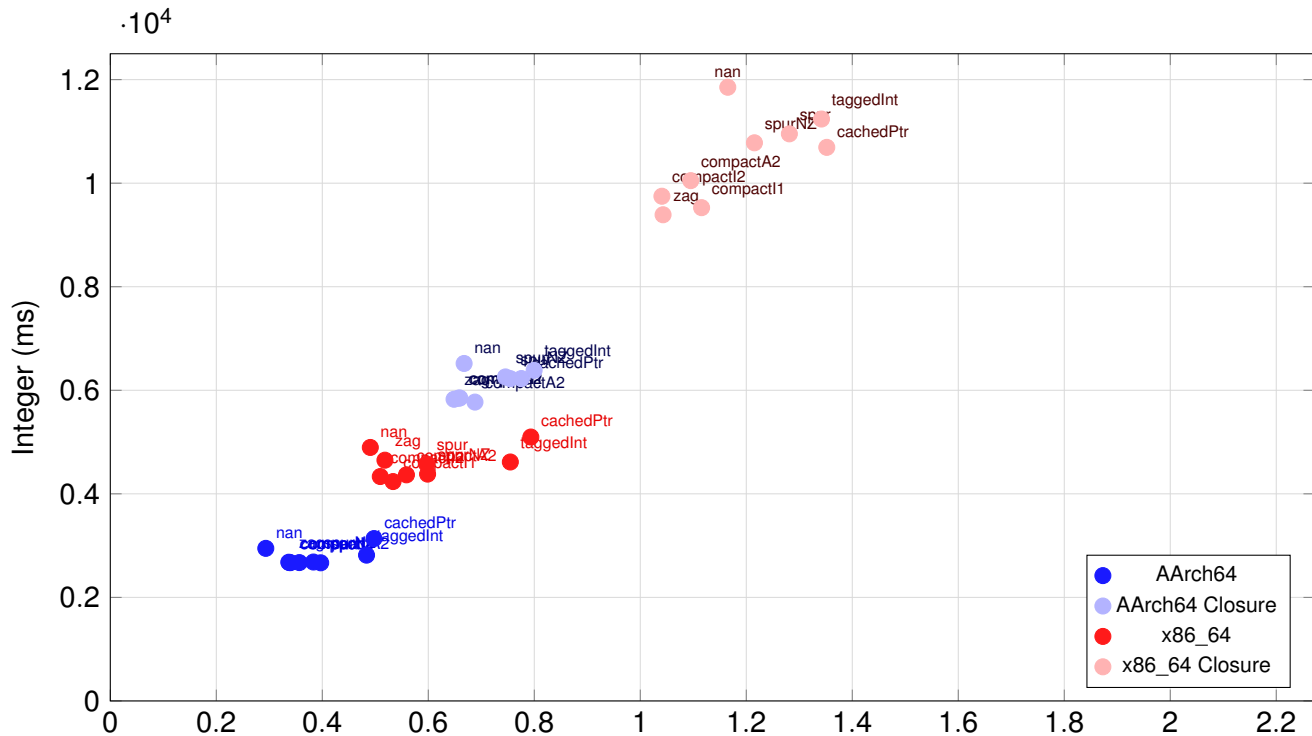
Optimized & BlockClosures - AARCH64



Optimized & BlockClosures - X86-64



Integer vs Float



Limitations

- Zag design features other than the choice of encoding will influence some of the runtimes
- In particular, Zag has a three-tier memory system: stack, nursery, and global heap, and references are only allowed to go to the right, but none of the test runs should hit the global heap, and the nursery is a copying collector, so should be quite efficient.
- Zag does no inlining of special selectors, so the “no inlining” versions will create actual *BlockClosure*s.
- The non-local return block is particularly tricky, because it references the context from which it must return, non-immediate closures can still be stack allocated, but create some extra work and stack pressure
- This may lead to significant churn in the first cases which have no immediate booleans or closures
- The encodings from Nan to Zag create immediate values for this block, so do not force any context spilling, and may, in fact, create no nursery allocations at all.

- there is a large design space for object encoding
- can be a memory-first or an immediate-first design philosophy
- lots of potential implications
- **Future work** is required to qualify these implications

Questions?

`dmason@torontomu.ca`

`n1kapadia@torontomu.ca`

`https://github.com/Zag-Research/Zag-Smalltalk`